Appendix G: AI Usage Log

Student: Kamil Szlufik | 15377821 | 15377821@student.setu.ie

AI tool: Claude (Anthropic, 2025) — claude.ai

Period: January 2026 – March 2026

Overview

This appendix logs how I used Claude during the PlayPal project. I used it mainly for three things: debugging errors I couldn't track down from the docs alone, learning Flutter and Firebase concepts since both were new to me, and getting unstuck on specific implementation problems like the Steam CORS issue. I also ran sections of the written report through it to catch spelling mistakes and punctuation errors before submitting.

Everything below is a genuine example from the project — I picked the ones that show the range of how it was used. Blue boxes are my prompts. Green boxes are what the AI responded with, including any code. Orange boxes describe what I actually did with it.

Box colour	What it represents
Blue left border	My prompt to Claude
Green left border	Claude's response and any code it provided
Orange left border	What I actually did with it — my own work

1. Debugging

Most of these came up late at night when something stopped working and the error message wasn't giving me much to go on. I used Claude the same way I'd use Stack Overflow — describe the problem, see what comes back, figure out if it applies to my situation.

1.1 Firestore permission-denied right after account creation

My prompt

> getting a really annoying bug. when a new user signs up the firebase auth account
> gets created fine but then immediately after i try to write the user doc to firestore
> and i get permission-denied. my security rules say authenticated users can write to
> their own document. i checked the rules in the console they look right. what is going on

AI response

> The AI said the issue was a token propagation delay — Firebase Auth creates the account and resolves the Future but the Firestore backend hasn't received the auth token yet. So when the Firestore write happens in the next line, the rules engine sees it as unauthenticated and rejects it. The fix was a short delay between the two operations:

```
// sign_up_screen.dart

Future<void> signUp() async {
  final username = _usernameController.text.trim();
  final email    = _emailController.text.trim();

  setState(() { _isLoading = true; _errorMessage = null; });

  try {
    // Step 1 — create the Firebase Auth account
    final cred = await FirebaseAuth.instance.createUserWithEmailAndPassword(
      email: email,
      password: _passwordController.text,
    );

    final uid = cred.user!.uid;

    // Step 2 — wait for auth token to propagate to Firestore
    // without this the write below gets permission-denied
    await Future.delayed(const Duration(milliseconds: 2000));

    // Step 3 — check username is not taken
    final existing = await FirebaseFirestore.instance
        .collection('usernames')
        .doc(username.toLowerCase())
        .get();

    if (existing.exists) {
      await cred.user!.delete();  // roll back the auth account
      setState(() {
        _errorMessage = 'That username is already taken.';
        _isLoading = false;
      });
      return;
    }

    // Step 4 — write user profile
    await FirebaseFirestore.instance.collection('users').doc(uid).set({
      'uid': uid,
      'email': email,
      'username': username,
      'usernameLower': username.toLowerCase(),
      'createdAt': FieldValue.serverTimestamp(),
    });

    // Step 5 — reserve the username
    await FirebaseFirestore.instance
        .collection('usernames')
        .doc(username.toLowerCase())
        .set({'uid': uid});

    if (mounted) Navigator.pop(context);

  } on FirebaseAuthException catch (e) {
    setState(() => _errorMessage = e.message);
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}
```

What I did with this

Put this straight into sign_up_screen.dart. Also added the username rollback (deleting the auth account if the username is taken) which wasn't in the original version — that was my own addition. Tested by creating about ten accounts in a row. No more permission errors. Documented in section 8.2.

1.2 Activity feed returning empty results with no error

My prompt

```
my feed screen loads but shows nothing. i can literally see the documents
in the
firebase console. no error in flutter either. the query is:

  _db.collection('activity')
    .where('uid', whereIn: friendUids)
    .orderBy('timestamp', descending: true)
    .limit(30)
    .get()

friendUids definitely has values. what am i missing
```

AI response

Firestore requires a composite index when combining whereIn on one field with orderBy on a different field. Without the index it silently returns zero results — no error, no warning. Also flagged that whereIn has a hard 10 value limit so if a user has more than 10 friends the query silently breaks. Gave the chunking fix:

```
// feed_screen.dart

Future<void> _loadFeed() async {
  setState(() => _isLoading = true);
  try {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) return;

    final friendsSnap = await _db
        .collection('users')
        .doc(user.uid)
        .collection('friends')
        .get();

    final friendUids = friendsSnap.docs.map((d) => d.id).toList();
    if (friendUids.isEmpty) {
      setState(() { _activities = []; _isLoading = false; });
      return;
    }

    // whereIn max is 10 — chunk into groups of 10
    final List<Map<String, dynamic>> allActivities = [];
    for (int i = 0; i < friendUids.length; i += 10) {
      final chunk = friendUids.sublist(
        i,
        (i + 10 > friendUids.length) ? friendUids.length : i + 10,
      );
      // requires composite index: uid ASC + timestamp DESC
      final snap = await _db
          .collection('activity')
          .where('uid', whereIn: chunk)
          .orderBy('timestamp', descending: true)
          .limit(50)
          .get();
      allActivities.addAll(
        snap.docs.map((d) => d.data() as Map<String, dynamic>),
      );
    }

    // sort combined results client-side
    allActivities.sort((a, b) {
```

```
      final ta = (a['timestamp'] as Timestamp?)?.seconds ?? 0;
      final tb = (b['timestamp'] as Timestamp?)?.seconds ?? 0;
      return tb.compareTo(ta);
    });

    setState(() { _activities = allActivities; _isLoading = false; });
  } catch (e) {
    if (mounted) setState(() => _isLoading = false);
  }
}
```

What I did with this

Created the composite index in Firebase Console — there was actually a direct link to it in the Firestore error logs. Implemented the chunking loop in feed_screen.dart. Hit the same missing index problem again later with the forum posts (likeCount + timestamp) and knew what to do immediately. Section 8.6.

1.3 iPhone black screen after deployment

My prompt

```
deployed to firebase hosting. works perfectly on desktop chrome.
opened it on my iphone and just a black screen. nothing loads.
flutter web. what would cause a black screen specifically on mobile safari
```

AI response

Two likely causes: RenderFlex overflow errors in the nav bar causing Flutter's renderer to crash silently on mobile, and the CanvasKit renderer which has known issues on Safari. Recommended checking the browser dev tools console while emulating iPhone size to catch overflow errors. Fixes:

```
// home_screen.dart — responsive top bar for narrow screens

Widget _buildTopBar(bool isWideScreen) {
  final isNarrow = MediaQuery.of(context).size.width < 400;

  return Container(
    height: 56,
    padding: EdgeInsets.symmetric(horizontal: isNarrow ? 8 : 16),
    decoration: BoxDecoration(
      color: _surface,
      border: Border(bottom: BorderSide(color: _border)),
    ),
    child: Row(
      children: [
        if (!isWideScreen) ...[
          IconButton(
            icon: const Icon(Icons.menu, color: _accent, size: 22),
            onPressed: () => _scaffoldKey.currentState?.openDrawer(),
            padding: EdgeInsets.zero,
            constraints: const BoxConstraints(),
          ),
          SizedBox(width: isNarrow ? 4 : 8),
        ],
        Expanded(
          child: Text(
            'PlayPal',
            style: TextStyle(
              color: _textPrimary,
              fontSize: isNarrow ? 14 : 16,
              fontWeight: FontWeight.bold,
            ),
```

```
            overflow: TextOverflow.ellipsis,
          ),
        ),
        // mobile: 3 buttons only — the rest move to the drawer
        if (!isWideScreen) ...[
          _topBarButton(icon: Icons.person_outline, tooltip: 'Profile', onTap:
_goToProfile),
          SizedBox(width: isNarrow ? 2 : 4),
          _friendsButton(),
          SizedBox(width: isNarrow ? 2 : 4),
          _topBarButton(icon: Icons.logout_outlined, tooltip: 'Sign out',
              onTap: _confirmSignOut, color: _textSecondary),
        ] else ...[
          // desktop: all 6 buttons
          _topBarButton(icon: Icons.person_outline, ...),
          _topBarButton(icon: Icons.people_outline, ...),
          _topBarButton(icon: Icons.videogame_asset_outlined, ...),
          _topBarButton(icon: Icons.rate_review_outlined, ...),
          _topBarButton(icon: Icons.favorite_outline, ...),
          _topBarButton(icon: Icons.logout_outlined, ...),
        ],
      ],
    ),
  );
}

// web/index.html — force HTML renderer to avoid CanvasKit/Safari issues
<script>
  window.flutterConfiguration = { renderer: 'html' };
</script>
```

What I did with this

Applied both fixes. Also found the same overflow issue in login_screen.dart and sign_up_screen.dart — both had SizedBox(width: double.infinity) inside a Row which causes the infinite constraint crash. Replaced those with the actual screen width from MediaQuery. Tested on iPhone SE (375px) and iPhone 14. Both work. Section 8.8.

1.4 Avatar colour not persisting between sessions

My prompt

```
users can pick an avatar colour from a colour picker on the profile
screen.
it saves fine and shows correctly but when they log out and back in the
colour
resets to the default cyan. the colour is being saved to firestore i can
see it
in the console. where is the bug
```

AI response

The AI asked how the colour was being stored and read back. When I showed the code it spotted that the hex string was being stored with a # prefix but the parsing code was trying to parse it without stripping the # first — so int.parse was failing silently and falling back to the default colour. Fix:

```
// profile_screen.dart

// Saving the colour — store as hex string with # prefix
Future<void> _saveAvatarColor(Color color) async {
  // toRadixString(16) gives e.g. 'ff00e5ff'
  // substring(2) strips the alpha channel prefix
  final colorHex = '#${color.value.toRadixString(16).substring(2).toUpperCase()}'
```

```
  // stores as '#00E5FF'
  try {
    await _db.collection('users').doc(_uid).update({'avatarColor': colorHex});
    setState(() => _profile!['avatarColor'] = colorHex);
    _showSnack('Avatar colour updated!', _green);
  } catch (_) {}
}

// Reading the colour back — must strip # before parsing
Color _getAvatarColor() {
  final hex = _profile?['avatarColor'];
  if (hex == null) return _accent;  // fallback to default
  try {
    // replaceAll strips the # so int.parse gets '00E5FF'
    // 'FF' prefix adds full opacity since Flutter Color needs ARGB
    return Color(int.parse('FF${hex.replaceAll('#', '')}', radix: 16));
  } catch (_) {
    return _accent;  // fallback if parse fails for any reason
  }
}
```

What I did with this

> *The bug was exactly what the AI spotted — the # wasn't being stripped before parsing. One line fix in _getAvatarColor(). Tested by picking different colours, logging out, logging back in — persists correctly now.*

1.5 Password reset email not triggering success message

My prompt

```
the forgot password flow sends the email fine — i can see it arrive —
but the success message never shows on screen. no error either.
the button just goes back to its normal state. here is the function:

  Future<void> _resetPassword() async {
    final email = _emailController.text.trim();
    setState(() { _isLoading = true; });
    await FirebaseAuth.instance.sendPasswordResetEmail(email: email);
    setState(() => _successMessage = 'Email sent!');
  }
```

AI response

The finally block is missing. If the setState for the success message runs before _isLoading is set back to false, the loading spinner stays visible and covers the message. Also if an exception is thrown the error is silently swallowed. The corrected pattern:

```
// login_screen.dart

Future<void> _resetPassword() async {
  final email = _emailController.text.trim();
  if (email.isEmpty) {
    setState(() => _errorMessage = 'Please enter your email address first.');
    return;
  }
  setState(() {
    _isLoading = true;
    _errorMessage = null;
    _successMessage = null;
  });
  try {
    await FirebaseAuth.instance.sendPasswordResetEmail(email: email);
    if (mounted) {
```

```
      setState(() =>
        _successMessage = 'Password reset email sent to $email. Check your
inbox!',
      );
    }
  } on FirebaseAuthException catch (e) {
    setState(() => _errorMessage = e.message);
  } finally {
    // always runs — clears the spinner regardless of success or error
    if (mounted) setState(() => _isLoading = false);
  }
}
```

What I did with this

> *The finally block was the missing piece. Once that was in, the success message appeared correctly and the spinner cleared. Also added the empty email check at the top which was a useful addition — without it the Firebase call would fail with a confusing error.*

2. Learning New Technologies

Flutter and Dart were completely new to me. I'd done JavaScript and Python in the course but Dart's type system and Flutter's widget model are quite different. I used Claude a lot in the first few weeks just to understand how things worked before trying to implement anything.

2.1 How StatefulWidget and the widget lifecycle works

My prompt

```
coming from javascript. in flutter what is the difference between
StatelessWidget
and StatefulWidget. when do i use each. also explain initState setState
dispose
and what mounted means — i keep seeing if (mounted) before setState calls
and
i dont know why its there
```

AI response

The AI walked through the full lifecycle. Key points: initState runs once when the widget enters the tree — right place to load data. setState tells Flutter to rebuild with new values. dispose cleans up controllers and subscriptions when the widget leaves the tree. The mounted check exists because async functions can complete after the widget is removed — calling setState on a disposed widget crashes the app. Example:

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  List<Game> games = [];
  bool isLoading = true;
  final TextEditingController searchController = TextEditingController();

  @override
  void initState() {
    super.initState();
```

```
    // runs once when widget enters the tree
    // kick off data fetching here, not in build()
    fetchGames();
    loadFavorites();
  }

  Future<void> fetchGames() async {
    final result = await rawgService.fetchGames();
    // mounted check: if user navigated away while this was loading
    // calling setState on a dead widget crashes the app
    if (mounted) {
      setState(() {
        games = result.map((json) => Game.fromJson(json)).toList();
        isLoading = false;
      });
    }
  }

  @override
  void dispose() {
    searchController.dispose();  // always clean up controllers
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    // called every time setState runs — keep it fast
    if (isLoading) return Center(child: CircularProgressIndicator());
    return GameGrid(games: games, ...);
  }
}
```

What I learned

> *This clicked everything into place. Every screen in PlayPal uses this exact pattern. The mounted check before every setState became automatic after the first few screens. Before this explanation I was getting a lot of 'setState called after dispose' crashes that I didn't understand.*

2.2 Streams vs Futures and how StreamBuilder works

My prompt

```
i want my forum to update in real time when someone posts without the user
having to refresh. i know firestore has something for this.
whats the difference between a future and a stream and how does
streambuilder work in flutter
```

AI response

A Future gives you one value and finishes. A Stream keeps emitting values over time. Firestore's .snapshots() returns a Stream that fires every time the data changes. StreamBuilder is a Flutter widget that subscribes to a Stream and rebuilds the UI automatically on each new value. It handles three states — waiting, error, and data. Example:

```
// forum_screen.dart — real-time posts using StreamBuilder

Widget _buildPostList() {
  return StreamBuilder<QuerySnapshot>(
    stream: FirebaseFirestore.instance
        .collection('forums')
        .doc(widget.gameId.toString())
        .collection('posts')
        .orderBy('timestamp', descending: false)
        .snapshots(),  // .snapshots() returns Stream — fires on every change
```

```
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Center(child: SizedBox(
          width: 24, height: 24,
          child: CircularProgressIndicator(strokeWidth: 2),
        ));
      }
      if (snapshot.hasError) {
        return Center(child: Text('Could not load posts'));
      }

      final docs = snapshot.data?.docs ?? [];

      // find top post client-side for the trophy badge
      String? topPostId;
      int maxLikes = 0;
      for (final doc in docs) {
        final likes = (doc['likeCount'] as num?)?.toInt() ?? 0;
        if (likes > maxLikes) { maxLikes = likes; topPostId = doc.id; }
      }
      if (maxLikes == 0) topPostId = null;

      if (docs.isEmpty) return _buildEmptyState();

      return ListView.builder(
        shrinkWrap: true,
        physics: const NeverScrollableScrollPhysics(),
        itemCount: docs.length,
        itemBuilder: (context, index) {
          final doc  = docs[index];
          final data = doc.data() as Map<String, dynamic>;
          return _buildPostCard(data, doc.id, doc.id == topPostId);
        },
      );
    },
  );
}
```

What I learned

Used StreamBuilder in two places — the forum screen and auth_gate.dart which listens to authStateChanges() to automatically route between login and home. The real-time forum was probably my favourite feature once it was working. New posts just appear without any refresh needed.

2.3 Understanding Firestore security rules

My prompt

```
how do firestore security rules work. i need users to only be able to read
and
write their own data but i also need any authenticated user to be able to
write
a friend request to another user's friendRequests subcollection.
can you show me how to write these rules
```

AI response

The AI explained the rules syntax and wrote the full rule set needed for PlayPal — the tricky part being the friendRequests exception where a user needs to write to someone else's subcollection:

```
rules_version = '2';
service cloud.firestore {
```

```
  match /databases/{database}/documents {

    // username lookup — any authed user can read/write
    // needed so users can check availability during signup
    match /usernames/{username} {
      allow read, write: if request.auth != null;
    }

    match /users/{userId} {
      // any logged-in user can read profiles (for friend search)
      allow read: if request.auth != null;
      // only owner can write their own profile
      allow write: if request.auth != null && request.auth.uid == userId;

      // all subcollections default to owner-only
      match /{subcollection=**} {
        allow read: if request.auth != null;
        allow write: if request.auth != null && request.auth.uid == userId;
      }

      // exception: any authed user can send a friend request
      // this allows writing to ANOTHER user's friendRequests
      match /friendRequests/{fromId} {
        allow write: if request.auth != null;
        allow read:  if request.auth != null && request.auth.uid == userId;
      }
    }

    // global activity feed
    match /activity/{activityId} {
      allow read, write: if request.auth != null;
    }

    // forums — anyone can post, only author can delete
    match /forums/{gameId}/posts/{postId} {
      allow read:   if request.auth != null;
      allow create: if request.auth != null;
      allow update: if request.auth != null;
      allow delete: if request.auth != null
        && request.auth.uid == resource.data.uid;
    }
  }
}
```

What I did with this

> *Published these rules to Firebase and tested them manually — tried writing to another user's favourites subcollection from a different account and got permission-denied as expected. The friendRequests exception was the part I couldn't figure out from the docs alone — the explanation of why you need to explicitly override the wildcard rule made it clear. Rules are in Appendix D.*

3. Implementation Help

These were situations where I had a clear idea of what I wanted to build but hit a specific technical wall. In each case the feature design and requirements were mine — I just needed help getting past a blocker.

3.1 Steam API CORS — routing through Cloud Functions

My prompt

```
i want users to connect their steam account and import their library.
when i call the steam web api directly from flutter web i get a cors
error.
the steam api doesn't support cors headers. i already use firebase.
how do i get around this
```

AI response

The Steam API doesn't set Access-Control-Allow-Origin so browsers block the request before it even reaches Steam. The fix for a Firebase project is to proxy all Steam calls through Cloud Functions — they run Node.js server-side with no CORS restrictions. The Flutter client calls the function over HTTPS, the function calls Steam, returns the result. Also means the API key never touches the client. Example:

```
// functions/index.js

const { onRequest } = require('firebase-functions/v2/https');
const { defineSecret } = require('firebase-functions/params');
const axios = require('axios');

const STEAM_API_KEY = defineSecret('STEAM_API_KEY');

exports.getSteamProfile = onRequest(
  { secrets: [STEAM_API_KEY], cors: true },
  async (req, res) => {
    const { steamId } = req.query;
    const apiKey = STEAM_API_KEY.value();

    const response = await axios.get(
      'https://api.steampowered.com/ISteamUser/GetPlayerSummaries/v2/',
      { params: { key: apiKey, steamids: steamId } }
    );

    const player = response.data?.response?.players?.[0];
    if (!player) { res.status(404).json({ error: 'Profile not found' }); return; }

    res.json({
      username:    player.personaname,
      avatar:      player.avatarfull,
      countryCode: player.loccountrycode ?? '',
    });
  }
);

exports.getSteamGames = onRequest(
  { secrets: [STEAM_API_KEY], cors: true },
  async (req, res) => {
    const { steamId } = req.query;
    const response = await axios.get(
      'https://api.steampowered.com/IPlayerService/GetOwnedGames/v1/',
      { params: {
          key: STEAM_API_KEY.value(), steamid: steamId,
          include_appinfo: true, include_played_free_games: true
      }}
    );
    const games = response.data?.response?.games ?? [];
    res.json({
      totalGames:  games.length,
      playedGames: games.filter(g => g.playtime_forever > 0).length,
      totalHours:  Math.round(games.reduce((s,g) => s + g.playtime_forever, 0) /
60),
      games: games.map(g => ({
        appId:         g.appid,
        name:          g.name,
        playtimeHours: Math.round((g.playtime_forever || 0) / 60),
        headerUrl:
```

```
`https://cdn.akamai.steamstatic.com/steam/apps/${g.appid}/header.jpg`,
      })).sort((a, b) => b.playtimeHours - a.playtimeHours),
    });
  }
);

// steam_service.dart — Flutter calls the function
Future<Map<String, dynamic>> getSteamGames(String steamId) async {
  final uri = Uri.parse(
    'https://us-central1-playpal-app-123.cloudfunctions.net/getSteamGames'
    '?steamId=$steamId',
  );
  final response = await http.get(uri);
  if (response.statusCode == 200) {
    return json.decode(response.body) as Map<String, dynamic>;
  }
  throw Exception('Steam request failed: ${response.statusCode}');
}
```

What I did with this

Built three Cloud Functions — getSteamProfile, getSteamGames, getSteamAchievements. The achievements one was harder than the example because it needs to call two Steam endpoints in parallel and merge the results (one for unlock data, one for display names and icons). Wrote that merge logic myself using Promise.all in Node.js. The Steam ID parsing to handle full URLs, vanity names, and numeric IDs was also mine — took a fair bit of trial and error. Section 8.5 and Appendix E.

3.2 Adult content filtering in the RAWG service

My prompt

```
my game grid is showing adult games which is not great for a project
showcase.
rawg clearly has a lot of this content.
how do i reliably filter it out — not just sometimes but reliably
```

AI response

Use multiple layers — no single filter is 100% reliable. The most effective one is the Metacritic score filter because adult games almost never have a Metacritic score. Stack it with the ESRB rating filter and a client-side check for full reliability:

```
// rawg_service.dart

Future<List<dynamic>> fetchGames({
  int page = 1,
  String sortBy = '-rating',
  String genre = '',
  String search = '',
  String platform = '',
  String dates = '',
}) async {
  final queryParams = {
    'key': _apiKey,
    'page_size': '40',
    'page': page.toString(),
    'ordering': sortBy,

    // layer 1: ESRB — API level, excludes Adults Only at source
    'esrb_rating': 'everyone,everyone-10-plus,teen,mature',

    // layer 2: Metacritic required — most effective filter in practice
```

```
    // adult games almost never have Metacritic scores
    'metacritic': '1,100',

    // layer 3: no DLC or add-ons
    'exclude_additions': 'true',

    if (genre.isNotEmpty)    'genres':    genre,
    if (search.isNotEmpty)   'search':    search,
    if (platform.isNotEmpty) 'platforms': platform,
    if (dates.isNotEmpty)    'dates':     dates,
  };

  final uri = Uri.https('api.rawg.io', '/api/games', queryParams);
  final response = await http.get(uri);

  if (response.statusCode == 200) {
    final data    = json.decode(response.body);
    final results = data['results'] as List<dynamic>;

    // layer 4: client-side safety net
    final filtered = results.where((game) {
      final esrb = game['esrb_rating'];
      if (esrb == null) return true;
      return (esrb['slug'] ?? '') != 'adults-only';
    }).toList();

    return filtered;
  } else {
    throw Exception('RAWG API error: ${response.statusCode}');
  }
}
```

What I did with this

> *Implemented all four layers in rawg_service.dart. Manually browsed the grid as a new user after deploying — no inappropriate content visible. The Metacritic filter was by far the most effective one which surprised me. Documented in section 8.3.*

3.3 Saving and displaying recently viewed games

My prompt

```
i want a recently viewed games row at the top of the home screen.
it should show the last 10 games the user tapped on.
how do i save this to firestore and keep it capped at 10 items
```

AI response

Store a recentlyViewed subcollection under each user. Each document uses the gameId as the document ID so there are no duplicates — if the user views the same game twice it just overwrites the existing document. To cap at 10, query by timestamp and delete any documents beyond the limit after writing. Or use a simpler approach — query the 10 most recent by timestamp and that's all that ever displays:

```
// user_service.dart

Future<void> saveRecentlyViewed(Game game) async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;

  // use gameId as doc ID — prevents duplicates automatically
  await FirebaseFirestore.instance
      .collection('users')
```

```
      .doc(user.uid)
      .collection('recently Viewed')
      .doc(game.id.toString())
      .set({
        'gameId':          game.id,
        'name':            game.name,
        'backgroundImage': game.backgroundImage,
        'rating':          game.rating,
        'released':        game.released,
        'timestamp':       FieldValue.serverTimestamp(),
      });
}

Future<List<Game>> loadRecently Viewed() async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return [];

  final snap = await FirebaseFirestore.instance
      .collection('users')
      .doc(user.uid)
      .collection('recently Viewed')
      .orderBy('timestamp', descending: true)
      .limit(10)  // cap at 10 — older entries ignored but not deleted
      .get();

  return snap.docs.map((doc) {
    final d = doc.data();
    return Game(
      id:              d['gameId'] as int,
      name:            d['name'] as String,
      backgroundImage: d['backgroundImage'] as String,
      rating:          (d['rating'] as num).toDouble(),
      released:        d['released'] as String,
    );
  }).toList();
}
```

What I did with this

> *Implemented this in user_service.dart. The document ID as gameId approach was clever — didn't think of that myself. Means revisiting the same game just updates the timestamp and it floats to the top of the list rather than creating a duplicate. The recently viewed strip shows immediately after the first game tap.*

4. Report and Documentation

A few times during the write-up I used Claude to look something up before writing about it, and to proofread sections for spelling and punctuation before submission.

4.1 Proofreading report sections for spelling and punctuation

My prompt

```
can you proofread this section of my report and fix any spelling mistakes,
punctuation errors or awkward sentences. dont change the meaning or
rewrite
anything, just fix actual errors:

[pasted section of report text]
```

AI response

> The AI returned the same text with corrections highlighted — things like 'platoform' corrected to 'platform', missing commas in long sentences, 'Steam Web api' corrected to 'Steam Web API', and 'technical decisin' corrected to 'technical decision'. It flagged each change with a brief note explaining what was wrong.

What I did with this

> *Went through each correction individually and accepted the ones I agreed with. Did not accept rewrites — if the AI suggested changing a sentence structure I ignored it. The goal was fixing typos and missing punctuation, not changing the writing. Ran about six sections through this process before submission.*

4.2 Checking the RAMP methodology description was accurate

My prompt

```
i need to describe the RAMP methodology in my report. can you explain what
it
stands for and how each phase works. i want to make sure im describing it
accurately before i write the section
```

AI response

> RAMP stands for Research, Analysis, Model, Plan. It is a structured approach to software development used in academic project contexts where documented justification matters as much as the technical output. Research covers investigating the problem domain. Analysis turns findings into requirements using techniques like MoSCoW. Model covers architecture and design. Plan covers sprint structure, timelines and risk analysis. The AI noted the phases are rarely perfectly sequential in practice.

What I did with this

> *Used this to verify section 3 was accurate before submitting. Wrote the section myself in my own words — the AI's explanation was just a fact check. The note about phases not being sequential was something I mentioned in the report because it was true of how the project actually went.*

4.3 MoSCoW citation in Harvard format

My prompt

```
who created the MoSCoW prioritisation technique and when.
i need to cite it in harvard referencing style for my report
```

AI response

> MoSCoW was introduced by Dai Clegg in 1994. Standard citation from the book Case Method Fast-Track: A RAD Approach by Clegg and Barker (1994), Addison-Wesley. Harvard format: Clegg, D. and Barker, R. (1994). Case Method Fast-Track: A RAD Approach. Addison-Wesley.

What I did with this

> *Used this exact citation wherever MoSCoW appears in the report. Listed in the references section as Clegg and Barker (1994).*

Summary

Looking back honestly — Claude probably saved me somewhere between 20 and 30 hours of debugging time over the course of the project, mostly on things like the Firestore token delay and the composite index issues that would have taken ages to track down from the docs. It was also genuinely useful for getting up to speed with Flutter quickly in the first few weeks.

What it didn't do: design the app, choose the features, write the code, or write the report. Every decision about what PlayPal should do and how it should look was mine. The AI was a tool I used to get past specific blockers and check specific facts — not something that drove the project.

Category	Examples	My role
Debugging	Token delay, composite index, iPhone black screen, avatar colour, password reset	Identified symptoms, understood the fix, implemented and tested
Learning Flutter/Firebase	StatefulWidget lifecycle, Streams/StreamBuilder, security rules	Applied concepts throughout — every screen uses these patterns
Implementation	Steam CORS proxy, content filtering, recently viewed, forum streams	Provided the requirements; wrote, tested and debugged all code
Report and docs	Proofreading, RAMP accuracy check, MoSCoW citation	Used AI as a fact check and spell checker only